

Intermediate level

1. Web scraper

A web scraper is a tool or program that automatically extracts data from websites by parsing their HTML content. It is commonly used to collect information such as headings, links, or product details for analysis, reporting, or automation purposes.

Code Snippet:

```
#Intermediate Level
#1.Web Scraper
import requests
from bs4 import BeautifulSoup

def scrape_headings(url):
    try:

        response = requests.get(url)
        response.raise_for_status() # Raise error if the request fails

        soup = BeautifulSoup(response.text, "html.parser")

        headings = []
        for tag in ['h1', 'h2', 'h3']:
            for heading in soup.find_all(tag):
                headings.append(heading.get_text(strip=True))

        return headings

    except requests.exceptions.RequestException as e:
        print(f"Error fetching data: {e}")
        return []

# Example usage
if __name__ == "__main__":
    url = input("Enter website URL: ")
    headings = scrape_headings(url)

    if headings:
        print("\nHeadings found:")
        for i, heading in enumerate(headings, 1):
            print(f"{i}. {heading}")
    else:
        print("No headings found or unable to fetch data.")
```

Output:

Enter website URL: <https://www.python.org>

Headings found:

- 1.
2. Functions Defined
3. Compound Data Types
4. Intuitive Interpretation
5. All the Flow You'd Expect
6. Quick & Easy to Learn
7. Get Started
8. Download
9. Docs
10. Jobs
11. Latest News
12. Upcoming Events
13. Success Stories
14. Use Python for...
15. >>>Python Software Foundation

Description:

This Python script is a simple web scraper that extracts headings (h1, h2, h3) from a given website. It uses the requests library to fetch the HTML content and BeautifulSoup to parse and extract the heading text. The headings are displayed in a numbered list, and the code includes basic error handling for invalid URLs or network issues. This tool is useful for quickly analyzing webpage structure, though it does not capture dynamically generated content.

2.Hungman

Hangman is a classic word-guessing game where the player tries to guess a hidden word one letter at a time within a limited number of attempts. It is often used to improve vocabulary and spelling skills, providing hints or clues to make the guessing easier.

Code Snippet:

```

#Intermediate Level
#2.Hangman
import random

WORDS = {
    'python': 'A popular programming language',
    'hangman': 'The game you are playing',
    'cricket': 'A popular sport in India',
    'notebook': 'Used for Jupyter',
    'movie': 'An entertainment'
}

def play():
    word, hint = random.choice(list(WORDS.items()))
    guessed = set()
    attempts = 6

    print("Welcome to Hangman!")
    print("Hint:", hint)

    while attempts > 0:
        display = ' '.join([c if c in guessed else '_' for c in word])
        print("\nWord:", display)

        if all(c in guessed for c in word):
            print("You won! The word was:", word)
            return

    guess = input("Guess a letter: ").strip().lower()

```

```

    guess = input("Guess a letter: ").strip().lower()

    if len(guess) != 1 or not guess.isalpha():
        print("Please enter a single valid letter.")
        continue

    if guess in guessed:
        print("You already guessed that letter!")
        continue

    if guess in word:
        guessed.add(guess)
        print("Good guess!")
    else:
        attempts -= 1
        print("Wrong! Attempts left:", attempts)

```

```

        if guess in word:
            guessed.add(guess)
            print("Good guess!")
        else:
            attempts -= 1
            print("Wrong! Attempts left:", attempts)

    print("You lost! The word was:", word)

if __name__ == '__main__':
    |
    keys = list(WORDS.keys())
    random.shuffle(keys)
    for key in keys:
        play()
        print("\n--- Next Game ---\n")

```

Output:

```

Welcome to Hangman!
Hint: The game you are playing

Word: _ _ _ _ _
Guess a letter:  c
Wrong! Attempts left: 5

Word: _ _ _ _ _
Guess a letter:  r
Wrong! Attempts left: 4

Word: _ _ _ _ _
Guess a letter:  i
Wrong! Attempts left: 3

```

Word: _ _ _ _ _

Guess a letter: c

Wrong! Attempts left: 2

Word: _ _ _ _ _

Guess a letter: k

Wrong! Attempts left: 1

Word: _ _ _ _ _

Guess a letter: e

Wrong! Attempts left: 0

You lost! The word was: hangman

--- Next Game ---

Welcome to Hangman!

Hint: Used for Jupyter

Word: _ _ _ _ _

Guess a letter: n

Good guess!

Word: n _ _ _ _

Guess a letter: o

Good guess!

Word: n o _ _ _ o _

Guess a letter: t

Good guess!

Word: n o t _ _ o o _

Guess a letter: e

Good guess!

Word: n o t e _ o o _

Guess a letter: b

Good guess!

Word: n o t e b o o _

Guess a letter: o

You already guessed that letter!

Word: n o t e b o o _

Guess a letter: k

Good guess!

Word: n o t e b o o k

You won! The word was: notebook

--- Next Game ---

Welcome to Hangman!

Hint: An entertainment

Word: _ _ _ _ _

Guess a letter: c

Wrong! Attempts left: 5

Word: _ _ _ _ _

Guess a letter: e

Good guess!

Word: _ _ _ _ e

Guess a letter: n

Wrong! Attempts left: 4

Word: _ _ _ _ e

Guess a letter: i

Good guess!

Word: _ _ _ i e

Guess a letter: m

Good guess!

Word: _ _ _ _ _

Guess a letter: e

Good guess!

Word: _ _ _ _ e

Guess a letter: n

Wrong! Attempts left: 4

Word: _ _ _ _ e

Guess a letter: i

Good guess!

Word: _ _ _ i e

Guess a letter: m

Good guess!

Word: m _ _ i e

Guess a letter: a

Wrong! Attempts left: 3

Word: m _ _ i e

Guess a letter: o

Good guess!

Word: m o _ i e

Guess a letter: e

You already guessed that letter!

Word: m o _ i e

Guess a letter: v

Good guess!

Word: m o _ i e

Guess a letter: v

Good guess!

Word: m o v i e

You won! The word was: movie

--- Next Game ---

Description:

Hangman is a word-guessing game where the player tries to guess a randomly chosen word from a predefined list. The game initializes variables for the word, guessed letters, incorrect guesses, and allowed attempts, and provides a text-based interface showing the partially revealed word. Players input letters, which are validated against the word, and the game continuously checks for win or loss conditions. The game loop runs until the player either wins or loses, and an option to play again can be offered once a game concludes.